Документация программного комплекса для автоматического перебора параметров профилометра через Modbus

Разработчик

3 марта 2026 г.

Содержание

1 Введение **2**

2 Архитектура проекта **2**

3 Установка и запуск **2**
3.1 Требования 2
3.2 Установка зависимостей 3
3.3 Запуск 3

4 Описание модулей **3**
4.1 modbus.py 3
4.2 modbus_controller.py 3
4.3 experiment_runner.py 4
4.4 logger.py 4
4.5 gui.py 4
4.6 main.py 5

5 Файл конфигурации (config.json) **5**

6 Пример использования **5**

7 Возможные проблемы и их решение **6**
7.1 Порт не открывается 6
7.2 Устройство не отвечает 6
7.3 После остановки кнопки неактивны 6
7.4 Программа не запускается 6

8 Заключение **6**

1 Введение

Программный комплекс предназначен для автоматического изменения двух параметров профилометра (или любого Modbus-устройства) по заданным диапазонам с фиксацией результатов в лог-файл. Приложение имеет графический интерфейс, позволяющий управлять процессом, наблюдать за текущими значениями, прогрессом и при необходимости ставить эксперимент на паузу или останавливать.

Основные возможности:

- Запись значений в два Modbus-регистра хранения (функция 06).
- Генерация комбинаций значений из двух диапазонов с заданным шагом.
- Логирование каждой итерации в CSV-файл с временной меткой и флагами успеха.
- Управление экспериментом: старт, пауза, продолжение, остановка.
- Отображение текущих параметров, прогресса выполнения, количества записей.
- Окно лога событий для отслеживания ошибок и сообщений.
- Загрузка и сохранение конфигурации в JSON-файл.

2 Архитектура проекта

Проект состоит из следующих модулей:

main.py — точка входа, загружает конфигурацию, инициализирует GUI.

gui.py — графический интерфейс на Tkinter.

experiment_runner.py — поток, выполняющий перебор комбинаций.

modbus_controller.py — обёртка для работы с библиотекой Modbus.

modbus.py — предоставленная библиотека для работы с Modbus RTU.

logger.py — логирование результатов в CSV-файл.

config.json — файл конфигурации.

Схема взаимодействия модулей представлена на рисунке 1.

GUI (gui.py) ↔ ExperimentRunner ↔ ModbusController ↔ modbus.py
↕
Logger (logger.py) → CSV-файл

Рис. 1: Схема архитектуры

3 Установка и запуск

3.1 Требования

- Python 3.6 или выше.
- Библиотеки: pymodbus, pyserial (обычно входят в pymodbus), tkinter (входит в стандартную поставку Python).

3.2 Установка зависимостей

Создайте файл `requirements.txt` со следующим содержимым:

```
pymodbus>=3.0.0
pyserial>=3.5
```

Установите зависимости:

```
pip install -r requirements.txt
```

3.3 Запуск

```
python main.py [-c config.json]
```

По умолчанию используется файл `config.json` в текущей директории.

4 Описание модулей

4.1 modbus.py

Предоставляет функции:

- `write_modbus_register(device_address, register_address, value, port, baudrate, timeout)` — запись одного регистра.
- `write_multiple_registers(...)` — запись нескольких регистров (не используется в текущей версии).
- `diagnose_connection(port, baudrate)` — диагностика соединения.
- Класс `ModbusResponseFormatter` для форматирования ответов.

Возвращает кортеж `(success, data, error_code, formatted_message)`.

4.2 modbus_controller.py

Класс `ModbusController` инкапсулирует логику повторных попыток и предоставляет методы:

- `__init__(config)` — принимает словарь конфигурации.
- `write_param1(value)` — запись в регистр параметра 1.
- `write_param2(value)` — запись в регистр параметра 2.
- `diagnose()` — проверка соединения.
- `_write_with_retry(register, value)` — внутренний метод с повторами.

Количество попыток и задержка между ними задаются в конфиге (поля `retries`, `retry_delay`).

4.3 experiment_runner.py

Класс `ExperimentRunner` (наследник `threading.Thread`) выполняет перебор комбинаций в фоновом режиме. Параметры:

- `config` — словарь с настройками.
- `logger` — объект логгера (с методом `log`).
- `gui_callback` — функция обратного вызова для обновления GUI.

Методы управления:

- `pause()` — приостановить после текущей итерации.
- `resume()` — возобновить.
- `stop()` — остановить эксперимент.
- `is_paused()` — проверка состояния.

В методе `run()` генерируются списки значений из диапазонов, затем в двойном цикле выполняется запись параметров и логирование.

4.4 logger.py

Класс `CSVLogger`:

- `__init__(filename)` — создаёт файл и записывает заголовки.
- `log(param1, param2, success1, success2)` — добавляет строку с временной меткой.

Файл создаётся в кодировке UTF-8.

4.5 gui.py

Класс `ModbusExperimentGUI` реализует главное окно:

- Поля для редактирования параметров подключения и диапазонов.
- Индикаторы текущих значений параметров, прогресс-бар, счётчик записей.
- Кнопки управления: Старт, Пауза/Продолжить, Стоп.
- Область лога событий.
- Поле для выбора файла лога.

Методы:

- `start_experiment()` — создаёт логгер и поток эксперимента.
- `pause_experiment()` — переключает паузу.
- `stop_experiment()` — останавливает эксперимент.
- `update_progress()` — callback для обновления GUI из потока.

Для безопасности обновления виджетов используется `root.after()`.

4.6 main.py

Загружает конфигурацию, создаёт окно Tkinter и запускает главный цикл. Аргумент командной строки `-c` позволяет указать путь к конфигурационному файлу.

5 Файл конфигурации (config.json)

Пример:

```
{
    "port": "/dev/ttyUSB0",
    "baudrate": 9600,
    "device_address": 1,
    "register_1": 100,
    "register_2": 101,
    "range_1": {"min": 0, "max": 100, "step": 10},
    "range_2": {"min": 0, "max": 50, "step": 5},
    "delay_between_writes": 1.0,
    "retries": 3,
    "retry_delay": 0.5,
    "timeout": 2,
    "log_file": "data/log.csv"
}
```

Поля:

port — последовательный порт (например, COM3 на Windows).

baudrate — скорость передачи.

device_address — Modbus-адрес устройства.

register_1 , `register_2` — адреса регистров для параметров.

range_1 , `range_2` — объекты с полями `min`, `max`, `step`.

delay_between_writes — пауза между итерациями (сек).

retries — число повторных попыток при ошибке записи.

retry_delay — задержка между попытками (сек).

timeout — таймаут Modbus-запроса (сек).

log_file — путь к файлу лога (будет создан, если не существует).

6 Пример использования

1. Заполните конфигурационный файл в соответствии с вашим устройством.
2. Запустите программу: `python main.py`.
3. Убедитесь, что порт и параметры отображаются корректно.
4. При необходимости измените диапазоны или файл лога прямо в GUI.

5. Нажмите «Старт».
6. Наблюдайте за текущими значениями и прогрессом.
7. Для временной остановки нажмите «Пауза», для возобновления — «Продолжить».
8. По окончании или для досрочного завершения нажмите «Стоп».
9. После остановки можно запустить новый эксперимент с теми же или изменёнными настройками.

7 Возможные проблемы и их решение

7.1 Порт не открывается

- Проверьте права доступа к порту (на Linux: `sudo chmod 666 /dev/ttyUSB0` или добавьте пользователя в группу `dialout`).
- Убедитесь, что порт не занят другим приложением.
- Проверьте настройки в конфиге.

7.2 Устройство не отвечает

- Проверьте подключение проводов, питание.
- Убедитесь, что адрес устройства и адреса регистров верны.
- Увеличьте таймаут или количество повторных попыток.
- Запустите диагностику через `ModbusController.diagnose()`.

7.3 После остановки кнопки неактивны

- Это может быть связано с зависанием потока. В GUI предусмотрен принудительный сброс через 2 секунды. Если проблема повторяется, проверьте логи на наличие исключений.
- Убедитесь, что файл лога не заблокирован другой программой.

7.4 Программа не запускается

- Проверьте версию Python и установленные зависимости.
- Запустите из командной строки и посмотрите на сообщения об ошибках.

8 Заключение

Данный программный комплекс позволяет автоматизировать процесс подбора параметров Modbus-устройства с удобным графическим интерфейсом и детальным логированием. Архитектура модулей обеспечивает лёгкую модификацию и расширение функциональности. При возникновении вопросов обращайтесь к разработчику.